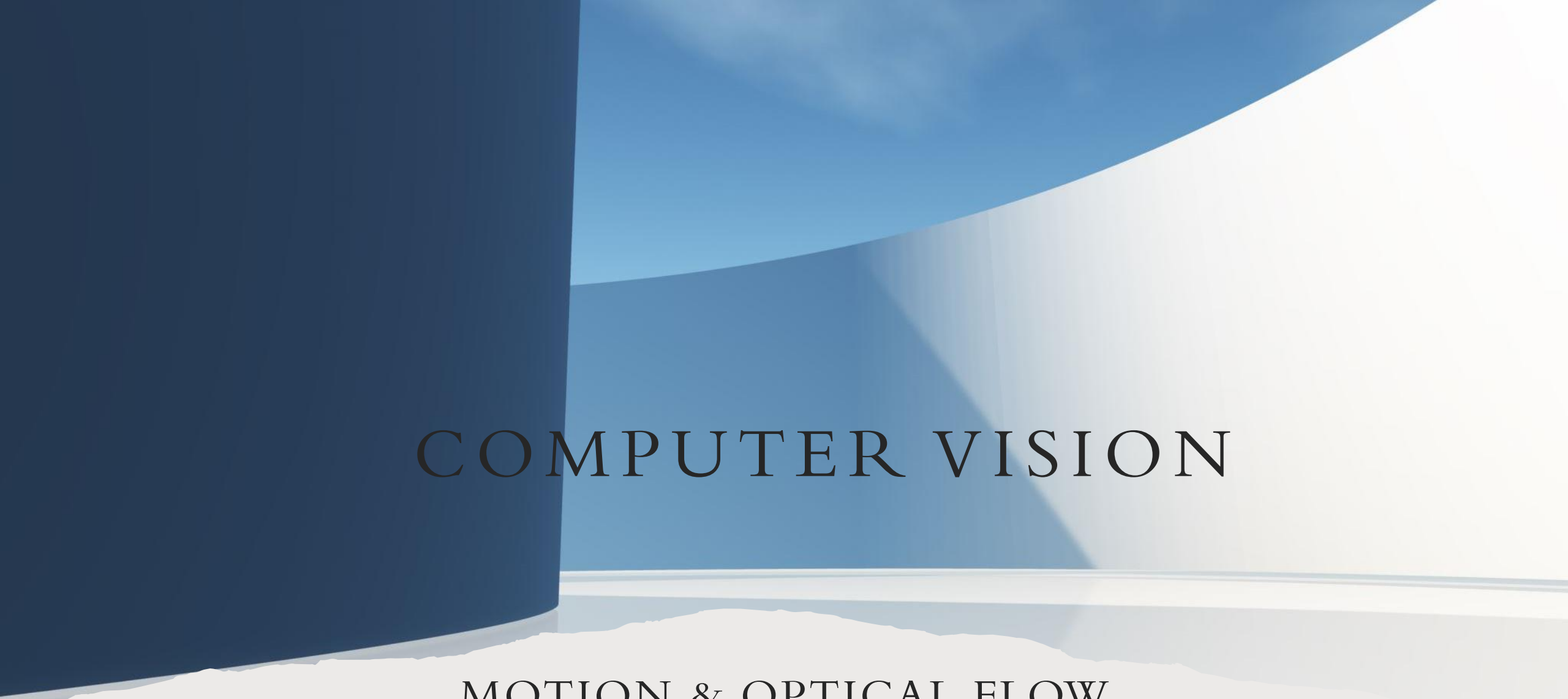


COMPUTER VISION

MOTION & OPTICAL FLOW

Ngo Quoc Viet-2025

CONTENTS

LECTURE OUTCOMES

- Understanding the concepts of motion estimation.
- Understanding and using optical flow algorithms such as Lucas-Kanade to estimate motion.
- Using functions like cv.calcOpticalFlowPyrLK() to track feature points in a video.
- Implementing simple action recognition based on optical flow.

MOTION ESTIMATION

- Motion refers to the change in position or appearance of objects in a sequence of images or frames over time.
- Analyzing motion is a fundamental aspect of computer vision, and it plays a crucial role in various applications, including video surveillance, object tracking, activity recognition, and robotics.
- There are several key concepts related to motion: Optical Flow, Motion Detection, Object Tracking, Activity Recognition
- It involves the analysis of consecutive frames in a video sequence to determine the motion of objects or the camera itself.

WHY MOTION ESTIMATION

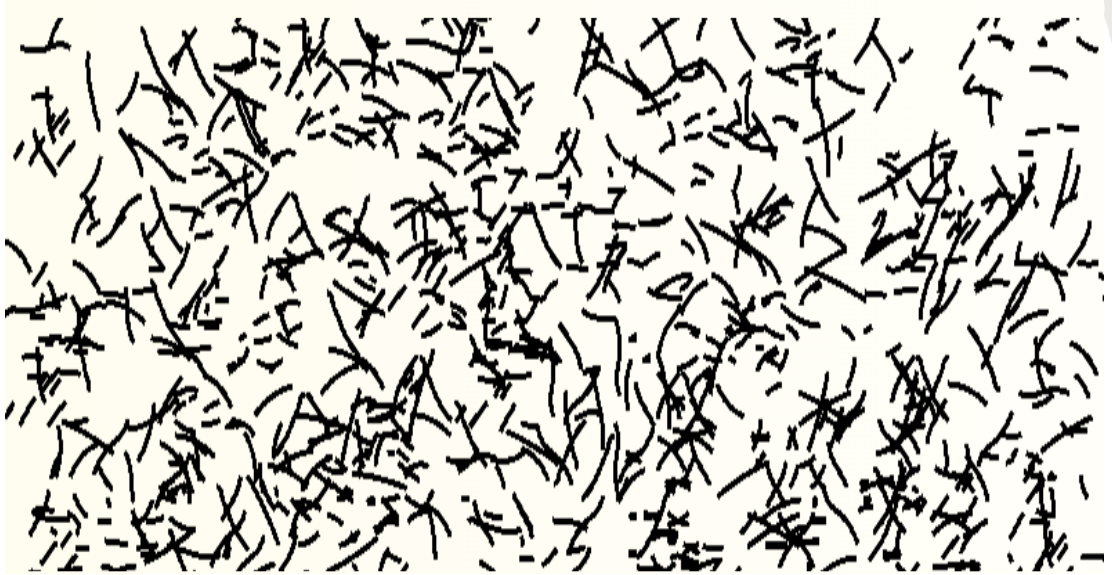
- **Video Compression**: Motion estimation helps in detecting changes between consecutive frames, allowing for more effective compression algorithms like MPEG standards.
- **Data Transmission and Storage**: It enables the transmission of only the changes in a scene, reducing the amount of data that needs to be transmitted or stored.
- **Object Tracking**: tracking the movement of objects within a scene is used in surveillance systems, autonomous vehicles, and robotics to follow the trajectory of objects or people.
- **VR and AR**: providing realistic and immersive experiences

HOW MOTION ESTIMATION

- **Block Matching**: simple but may not handle complex motions
- **Optical Flow**: estimate the motion of each pixel by analyzing the pattern of intensity changes between consecutive frames. The optical flow field represents the motion vectors for each pixel in the image. Various algorithms, such as Lucas-Kanade and Horn-Schunck.
- **Gradient-Based Methods**: computing the gradient of the image and using it to derive motion information. Lucas-Kanade algorithm.
- **Feature Tracking**: Algorithms like the Kanade-Tomasi tracker or the Shi-Tomasi
- **Deep Learning Approaches**.

MOTION FIELD

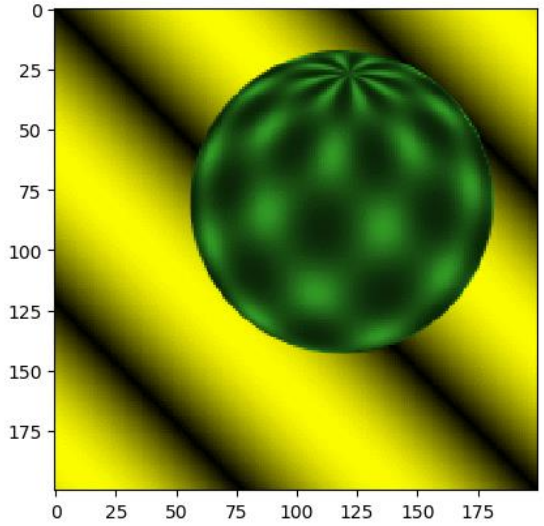
- Camera moves (rotation, translation)
- Objects in scence move rigidly
- Objects articulate (e.g: hand moving)
- Objects bend and deform
- Blowing smoke, clouds
- We estimate the motion field by computing the optical flow.

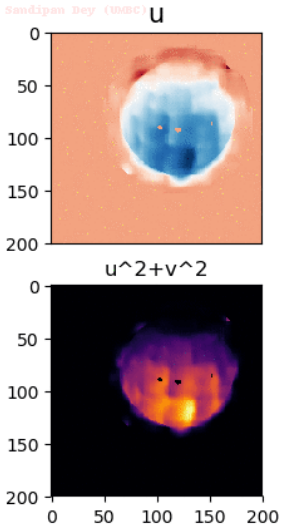


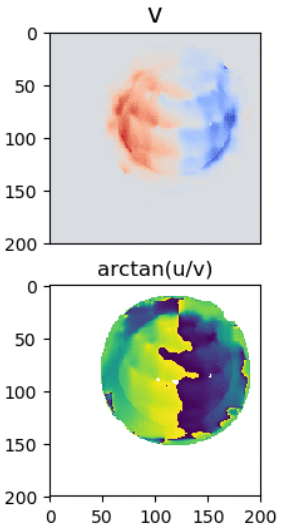
From: CSE 152, Spring 2018

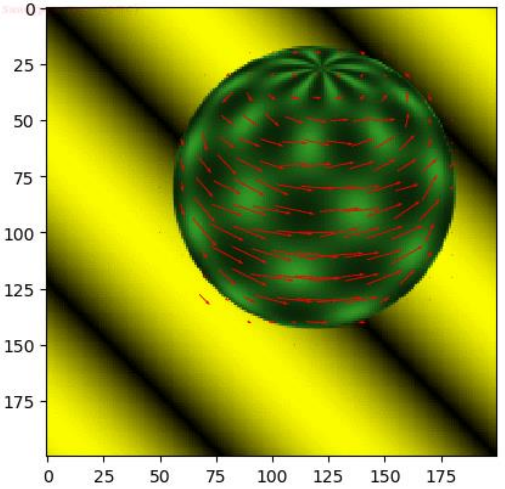
https://michaelbach.de/ot/cog-hiddenBird/index.html

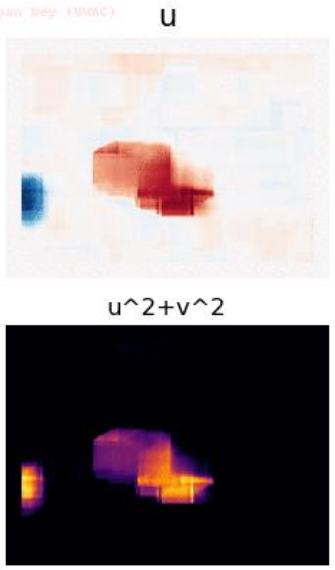
OPTICAL FLOW

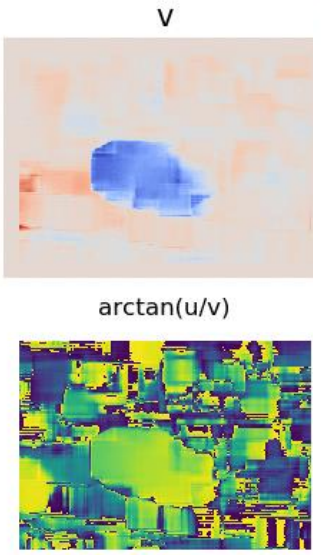


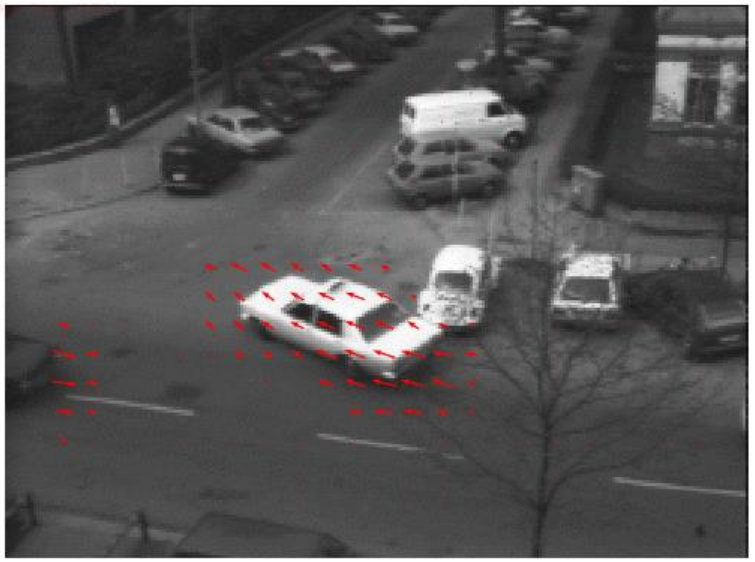










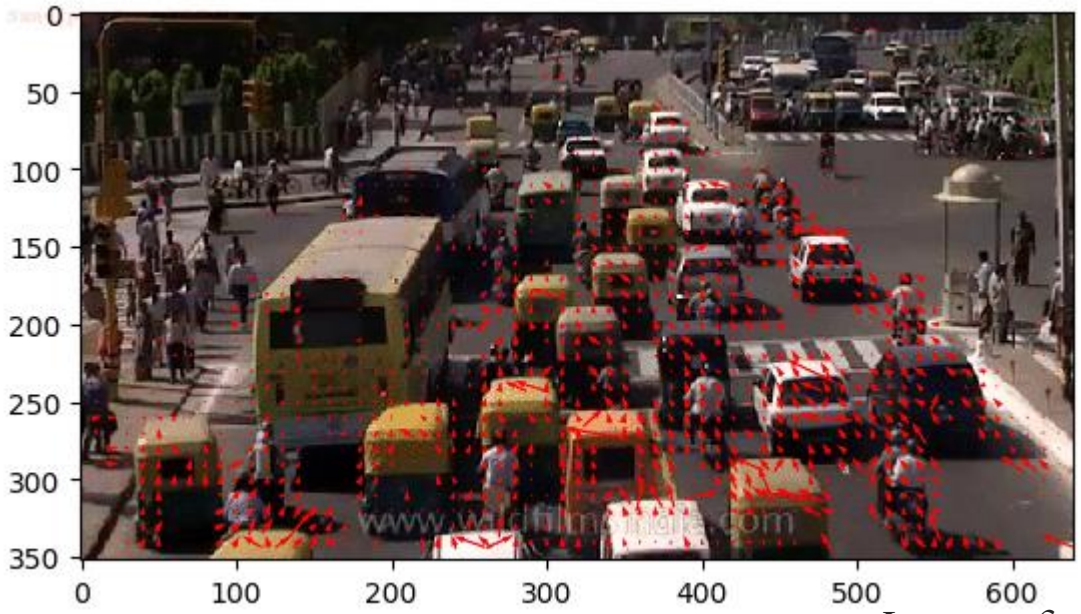


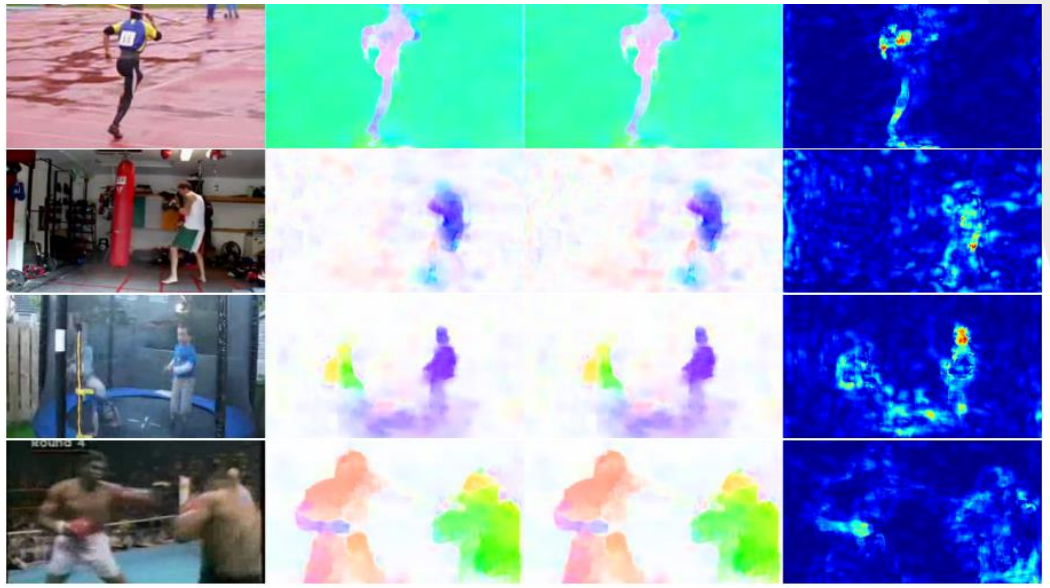
https://sandipanweb.wordpress.com/2018/02/25/implementing-lucas-kanade-optical-flow-algorithm-in-python/

OPTICAL FLOW

- The most general of motion estimation is to compute an independent estimate of motion at each pixel, which is generally known as optical (or optic) flow.
- Optical flow refers to the analysis of the apparent motion of objects or patterns within a visual scene based on the pixel intensity changes between consecutive frames of a video or image sequence.
- Optical flow is a vector field that shows the direction and magnitude of these intensity changes from one image to the other. These vectors convey information about the local motion patterns within the scene.
- The concept of optical flow is particularly valuable for tasks such as motion analysis, object tracking, and scene understanding. By computing the optical flow, computer vision systems can infer information about how objects are moving, their speed, and their direction of motion.
- The goal of optical flow algorithms is to **estimate the optical flow vector (u, v) for each pixel** in the image, satisfying this brightness constancy constraint.

WHY WE NEED OPTICAL FLOW





Images from: Sefidia Academy

Track the motion of cars/vehicles across frames to estimate its current velocity and predict its position

Recognize human actions

OPTICAL FLOW EQUATION

- Given an image $I(x, y, t)$ and move its pixels by (dx, dy) over t time, we obtain the new image $I(x + dx, y + dy, t + dt)$. Assume that the pixel intensities of an object are constant between consecutive frame, then

$$I(x, y, t) = I(x + dx, y + dy, t + dt)$$

- To solve above equation, we need to approximate the value of right hand by taking the Taylor series approximation. Taking first-order Taylor expansion at the neighbour of (x, y, t)

$$I(x + dx, y + dy, t + dt) \approx I(x, y, t) + \frac{\partial I}{\partial x}dx + \frac{\partial I}{\partial y}dy + \frac{\partial I}{\partial t}dt + \cdots$$

- Rearranging terms and setting the approximation equal to the original expression, we get

$$\frac{\partial I}{\partial x}dx + \frac{\partial I}{\partial y}dy + \frac{\partial I}{\partial t}dt \approx 0$$

- Dividing by *dt* to get optical flow equation

$$\frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \frac{\partial I}{\partial t} = \boldsymbol{I_x u + I_y v + I_t} = 0, u = \frac{dx}{dt}, v = \frac{dy}{dt}$$

$\partial I/\partial x$, $\partial I/\partial y$, and $\partial I/\partial t$ are the image gradients (derivative of intensity) along the horizontal axis, the vertical axis, and time

https://brilliant.org/wiki/taylor-series-approximation/

OPTICAL FLOW EQUATION

- The problem of optical flow, that is, **solving $u = \frac{dx}{dt}$ and $v = \frac{dy}{dt}$** to determine movement over time.

- We cannot directly solve the optical flow equation for u and v since there is only one equation for two unknown variables.

- We will implement some methods such as the Lucas-Kanade method to address this issue.

BRIGHTNESS CONSTANCY EQUATION

- The Brightness Constancy Equation is a fundamental assumption in optical flow estimation, i.e the intensity (brightness) of a pixel remains constant over time.
- Constancy Equation is expressed as

$$I_x u + I_y v + I_t = 0$$

  - I_x and I_y are the partial derivatives of image intensity with respect to the coordinates x, y. I_t is the temporal derivative of image intensity with respect to time t.
  - u, v are the horizontal and vertical components of the optical flow vector, representing the motion of the pixel in the image.
- Optical flow constraint or brightness constancy constraint equation (Horn and Schunck 1981)

OPTICAL FLOW METHODS

- The goal of optical flow algorithms is to **estimate the optical flow vector** (u, v) **for each pixel** in the image, satisfying this brightness constancy constraint.
- Optical flow works on several assumptions
  - The pixel intensities of an object do not change between consecutive frames
  - Neighbouring pixels have similar motion
- Sparse Optical Flow: This approach tracks specific features or interest points in the image, such as corners or key points, and estimates the motion vectors of these points.
- Dense Optical Flow: This method calculates motion vectors for every pixel in an image, providing a dense field of motion information across the entire image

SPARSE OPTICAL FLOW METHODS

- Lucas-Kanade: This method assumes that the motion is essentially constant in a local neighborhood. It solves a system of linear equations for each pixel, considering the gradient of the image and the temporal gradients.
- KLT (KLT Tracker): The Kanade-Lucas-Tomasi tracker is an extension of the Lucas-Kanade method. It uses the eigenvalues of the gradient matrix to determine the features that are best suited for tracking.
- Feature Tracking Algorithms: Various feature tracking algorithms, such as Shi-Tomasi or Harris corner detectors, can be used to identify key points in the image, and then the motion of these features is tracked over time.

DENSE OPTICAL FLOW METHODS

- Horn-Schunck: This method formulates optical flow as a global constraint problem, assuming that the motion is smooth across the entire image. It minimizes the global error between the observed flow and the estimated flow.
- Farneback: Farneback's algorithm is a polynomial expansion method that estimates dense optical flow by approximating the motion with a polynomial function. It's known for being computationally efficient.
- Deep Learning-Based Methods: Convolutional neural networks (CNNs) and other deep learning architectures have been employed for optical flow estimation. Networks like FlowNet use convolutional layers to directly predict optical flow from image pairs.
- Variational Methods: Variational methods formulate optical flow as an optimization problem and use energy minimization techniques to estimate the flow field.

DENSE VS. SPARSE METHODS

	Dense Optical Flow	Sparse Optical Flow
Objective	Estimate motion for every pixel in an image or within specified regions	Estimate motion for a selected set of keypoints or interest points in an image.
Output	A dense motion field, where motion vectors are calculated for all pixels.	Motion vectors only for specific points or keypoints chosen in the image.
Applications	Useful in applications requiring detailed motion information across the entire image, such as video analysis, frame interpolation, and dense motion segmentation	Suitable for scenarios where motion information for specific features or locations is sufficient, such as object tracking, feature tracking, and matching
Algorithms	Farnebäck Optical Flow, Horn-Schunck Method (Adapted for Dense Flow), Brox Optical Flow, PCA-Flow, DeepFlow, PWC-Net (Pyramidal Network for Optical Flow)	Lucas-Kanade, KLT (Kanade-Lucas-Tomasi) Tracker, Good Features to Track (GFTT), SURF (Speeded Up Robust Features), ORB (Oriented FAST and Rotated BRIEF), Feature Matching with SIFT or AKAZE

LUCAS-KANADE OPTICAL FLOW ALGORITHM

- LK algorithm is based on the idea of linearizing the image intensity variations in both spatial and temporal dimensions to estimate the local motion.
- LK algorithm involves finding the motion (u, v) that **minimizes** the **sum-squared error** of the **brightness constancy equations** for each pixel in a **window**.
- LK method formulates the following system of linear equations for each pixel in a **local window**.

LUCAS-KANADE OPTICAL FLOW ALGORITHM

- **Spatial Gradients**: Compute the spatial gradients I_x and I_y by using spatial filter (Prewitt, Sobel,… operators). For example, Prewitt kernel

$$H_x = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix} \; H_y = \begin{bmatrix} -1 & -1 \\ 1 & 1 \end{bmatrix}$$

$$I_x = H_x * \mathrm{I}$$

- **Temporal Gradient**: compute the temporal gradient I_t by taking the pixel-wise difference between the two consecutive frames.

$$I_t(x, y) = \frac{\partial I}{\partial t} = I(x, y, t+1) - I(x, y, t)$$

- **Windowed Intensity Values**: Choose a local window around a pixel

LUCAS-KANADE OPTICAL FLOW ALGORITHM

- Lucas-Kanade Equation: $I_x u + I_y v = -I_t \Leftrightarrow \begin{bmatrix} I_x & I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = -I_t$

$$\begin{bmatrix} I_x & I_y \end{bmatrix}^T \begin{bmatrix} I_x & I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = -\begin{bmatrix} I_x & I_y \end{bmatrix}^T I_t$$

$$\begin{bmatrix} u \\ v \end{bmatrix} = -(\begin{bmatrix} I_x & I_y \end{bmatrix}^T \begin{bmatrix} I_x & I_y \end{bmatrix})^{-1} \begin{bmatrix} I_x & I_y \end{bmatrix}^T \begin{bmatrix} I_x & I_y \end{bmatrix} I_t$$

- Consider $\mathrm{n} = kxk$ window (n pixels) arounding the specific pixel

$$\begin{bmatrix} I_{x1} & I_{y1} \\ I_{x2} & I_{y2} \\ \vdots & \vdots \\ I_{xn} & I_{yn} \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} -I_{t1} \\ -I_{t2} \\ \vdots \\ -I_{tn} \end{bmatrix}$$

$$A \begin{bmatrix} u \\ v \end{bmatrix} = I_t \Rightarrow A^T A \begin{bmatrix} u \\ v \end{bmatrix} = A^T I_t \Rightarrow \begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T I_t$$

- We solve a least-square problem to **find the best** flow vectors for the pixel

LUCAS-KANADE OPTICAL FLOW ALGORITHM

- Instead of one equation $[I_x \quad I_y]\begin{bmatrix}u\\v\end{bmatrix} = -I_t$ with two unknow variables, we have n linear equations (local window with n pixels). The problem of over-determined.

$$\begin{bmatrix} I_x(p_1) & I_y(p_1) \\ I_x(p_2) & I_y(p_2) \\ \vdots & \vdots \\ I_x(p_n) & I_x(p_n) \end{bmatrix}\begin{bmatrix}u\\v\end{bmatrix} = \begin{bmatrix} -I_t(p_1) \\ -I_t(p_2) \\ \vdots \\ -I_t(p_n) \end{bmatrix}$$

- In such cases, a least squares approach is used to find the best-fitting solution.
- The least squares fit provides a way to solve this system of equations and find the motion parameters that best satisfy the constraints imposed by the brightness constancy assumption.
- The least squares fit aims to minimize the sum of squared differences (residuals) between the observed intensity changes (temporal gradient I_t) and the predicted changes based on the motion parameters. This minimization process provides a robust estimation of the motion parameters that best fits the observed data.

LUCAS-KANADE OPTICAL FLOW ALGORITHM

- To address the over-determined issue, we apply least squares fitting to obtain the following two-equation-two-unknown problem

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \sum_i I_x(p_i)^2 & \sum_i I_x(p_i) I_y(p_i) \\ \sum_i I_y(p_i) I_x(p_i) & \sum_i I_y(p_i)^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum_i I_x(p_i) I_t(p_i) & -\sum_i I_y(p_i) I_t(p_i) \end{bmatrix}$$

Least squares fitting: https://mathworld.wolfram.com/LeastSquaresFitting.html; https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.leastsq.html.

LUCAS-KANADE STEP BY STEP

- We want to estimate the motion vector at $(1, 1)$ in 3x3 patch.
- Frame t: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$, t+1: $\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$.
- Using Iy, Ix = np.gradient(I1) to compute the spatial gradients: $I_x = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$, $I_y = \begin{bmatrix} 3 & 3 & 3 \\ 3 & 3 & 3 \\ 3 & 3 & 3 \end{bmatrix}$, Temporal Gradient $I_t = I = \begin{bmatrix} -1 & -1 & -1 \\ -1 & -1 & -1 \\ -1 & -1 & -1 \end{bmatrix}$,
- np.column_stack((Ix.flatten(), Iy.flatten())) to compute

$$A^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 3 & 3 & 3 & 3 & 3 & 3 & 3 & 3 & 3 \end{bmatrix}, b = -I_t = [1\ 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1]$$

- motion_params = np.linalg.lstsq(A, b, rcond=None)[0] to get $(u, v) = (0.1, 0.3)$

LUCAS-KANADE PYTHON STEP BY STEP

```python
def lucasKanade(I1, I2, x, y, winSize=15, tau=1e-2):
    # Extract a small image patch around the selected pixel
    halfwinSize = winSize // 2
    patch_1 = I1[y - halfwinSize:y + halfwinSize, x - halfwinSize:x + halfwinSize]
    patch_2 = I2[y - halfwinSize:y + halfwinSize, x - halfwinSize:x + halfwinSize]

    Ix, Iy = np.gradient(patch_1.astype(float))
    It = patch_2 - patch_1

    A = np.column_stack((Ix.flatten(), Iy.flatten()))
    b = -It.flatten()

    # Solve for motion parameters using least squares fit
    motion_params = np.linalg.lstsq(A, b, rcond=None)[0]

    # Extract motion components u and v
    u, v = motion_params

    return (u, v)
```

LUCAS-KANADE PYTHON WITH OPENCV

- Select features to track (SIFT, SURF, or else).
- while (cap.isOpened()):
  - next, status, error = cv.calcOpticalFlowPyrLK(prev_gray, gray, prev, None, **lk_params)
  - good_old = prev[status == 1]
  - # Selects good feature points for next position
  - good_new = next[status == 1]
  - Draw lines (motion)

- ret, first_frame = cap.read()
- prev_gray = cv.cvtColor(first_frame, cv.COLOR_BGR2GRAY)
- feature_params = dict(maxCorners = 300, qualityLevel = 0.2, minDistance = 2, blockSize = 7)
- prev = cv.goodFeaturesToTrack(prev_gray, mask = None, **feature_params)

- *lk_params = dict(winSize = (25,25), maxLevel = 2, criteria = (cv.TERM_CRITERIA_EPS | cv.TERM_CRITERIA_COUNT, 10, 0.03))*
- *next, status, error = cv.calcOpticalFlowPyrLK(prev_gray, gray, prev, None, **lk_params)*

PROBLEMS OF LUCAS-KANADE

- Brightness constancy is not satisfied.
  - The linear equations are derived from the first order Taylor Approximation.

$$I_x u + I_y v + I_t \approx 0$$

  - If the motion (u, v) is small, the first-order Taylor approximation is accurate. On the contrary, the first-order Taylor approximation is inaccurate.
- Suppose $A^T A$ is easily invertible.
  - The determinant of the matrix must be non-zero, and more importantly, both eigenvalues of the matrix must be large and of equal magnitude. It requires that movement can be clearly identified in both directions (the window in question is a corner or a rich textured area)
- The motion is not small
  - Coarse-to-fine optical flow estimation (Pyramid Lucas-Kanade, …) or Iterative LK to correct the erroneous model by reducing the large movement to a series of small movements, and at each step, Least Squares can operate efficiently
- A point does not move like its neighbors (window size is too large)

ACTION RECOGNITION WITH OPTICAL FLOW

- Optical can be used for motion analysis. It involves computing the flow vectors that represent the apparent motion of pixels between consecutive frames in a video sequence.
- Optical flow can be used to estimate the direction and speed of moving objects.
- OpenCV provides functions like cv2.calcOpticalFlowFarneback(), cv2.goodFeaturesToTrack() and cv2.calcOpticalFlowPyrLK() that can be used for **optical flow analysis, feature tracking**.
- Extract **pertinent information from the video frames** such as optical flow, or feature tracking to extract motion-related features from the video frames and feed them into machine learning model (SVM, AdaBoost, etc).
- Lucas-Kanade itself is not typically used for action recognition, it can be a component of a larger system for motion analysis.

ACTION RECOGNITION WITH OPTICAL FLOW

```python
def calculateOpticalFlowHistograms(opticalFlow, num_bins=8):
    # Calculate magnitudes and angles of optical flow vectors
    magnitudes, angles = cv2.cartToPolar(opticalFlow[:, :, 0], opticalFlow[:, :, 1])

    # Calculate histograms of optical flow magnitudes
    histograms = []
    for i in range(num_bins):
        bin_range = [i * (360 // num_bins), (i + 1) * (360 // num_bins)]
        hist, _ = np.histogram(angles, bins=num_bins, range=bin_range, weights=magnitudes)
        histograms.extend(hist)

    return np.array(histograms)
```

ACTION RECOGNITION WITH OPTICAL FLOW

```python
def actionFeaturesLK(videoPath):
    cap = cv2.VideoCapture(videoPath)
    # Read the first frame
    ret, prevFrame = cap.read()
    prevGray = cv2.cvtColor(prevFrame, cv2.COLOR_BGR2GRAY)
    prevPts = cv2.goodFeaturesToTrack(prevGray, mask=None, **featureParams)
    featureVectors = []
    while True:
        # Read the current frame
        ret, currentFrame = cap.read()
        if not ret: # Break the loop if the video has ended
            break
        # Convert frames to grayscale
        prevGray = cv2.cvtColor(prevFrame, cv2.COLOR_BGR2GRAY)
        currentGray = cv2.cvtColor(currentFrame, cv2.COLOR_BGR2GRAY)
        # Perform Lucas-Kanade optical flow
        prevPts, goodPoints = lucasKanadeOpticalFlow(prevGray, currentGray, prevPts)
        features = calculateOpticalFlowHistograms(goodPoints)
        # Append the feature vector to the list
        featureVectors.append(features)
        prevPts = goodPoints
        prevFrame = currentFrame.copy()
    # Convert the list of feature vectors to a NumPy array
    featureMatrix = np.array(featureVectors)
```

SOME CV2 FUNCTIONS FOR OPTIC FLOW

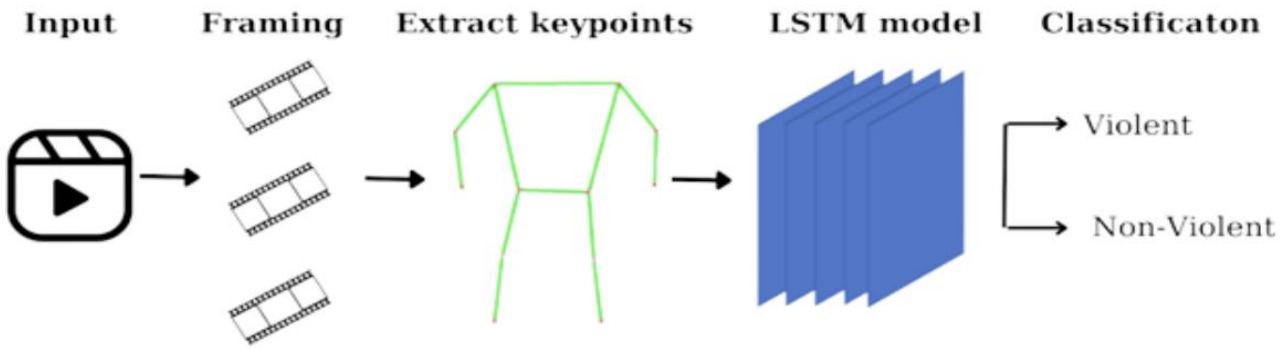
- calcOpticalFlowPyrLK (optical flow for a sparse feature set using the iterative Lucas-Kanade method with pyramids)
- calcOpticalFlowFarneback (dense optical flow using the Gunnar Farneback's algorithm).
- calcOpticalFlowHS (optical flow for two images using Horn-Schunck algorithm)

ACTIVITY RECOGNITION USING MEDIAPIPE

These demos are made by my student-Yen Thai Thi Kim.

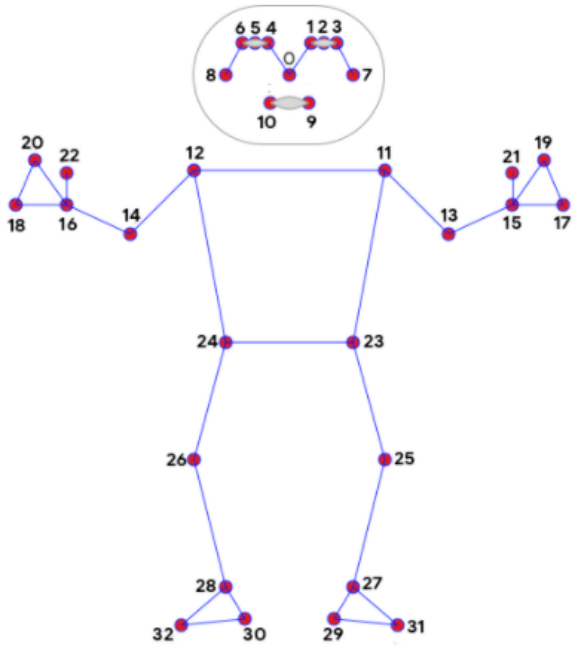
ACTIVITY RECOGNITION USING MEDIAPIPE

- This problem involves recognizing human actions with 3 classes: walking, playing cello and playing tennis. The data is collected in the form of videos from the following sources:



- The video features are from Google MediaPipe, https://github.com/google/mediapipe.
- Activity recognition is based from LSTM. For example: https://github.com/nam157/human_activity_recognition-.

ACTIVITY RECOGNITION USING MEDIAPIPE



0. nose
1. left_eye_inner
2. left_eye
3. left_eye_outer
4. right_eye_inner
5. right_eye
6. right_eye_outer
7. left_ear
8. right_ear
9. mouth_left
10. mouth_right
11. left_shoulder
12. right_shoulder
13. left_elbow
14. right_elbow
15. left_wrist
16. right_wrist
17. left_pinky
18. right_pinky
19. left_index
20. right_index
21. left_thumb
22. right_thumb
23. left_hip
24. right_hip
25. left_knee
26. right_knee
27. left_ankle
28. right_ankle
29. left_heel
30. right_heel
31. left_foot_index
32. right_foot_index

12 keypoints

- Each keypoint contains 4 parameters: x, y, z, visibility.
- With 12 keypoints per frame, a single frame captures 48 values.
- And there are 24 frames captured per second

SUMMARY

- Optical flow algorithms such as Lucas-Kanade to estimate motion.
- Lucas-Kanade algorithm is based on Brightness Constancy constraint equation and least squares method for neighbor of the specific location.
- The OpenCV functions for optical flow: goodFeaturesToTrackm calcOpticalFlowPyrLK, calcOpticalFlowFarneback, calcOpticalFlowHS.